

CS218 Tut-2

Question 1. Can you find square of an n -bit integer a , using square subroutine on five $n/3$ bit integers and a few additions/subtractions? You need to compute the following five terms using the square operation only 5 times.

$$P^2, PQ, 2PR + Q^2, QR, R^2.$$

Solution.

One useful observation we can make is that the 5 terms we need are just the coefficients of $(Px^2 + Qx + R)^2$ (if we just take $2^{n/3}$ as x). And to get the coefficients of a d degree polynomial, we just need to evaluate it at $d + 1$ different points. And since the degree is 4, this is perfect for us. Also to evaluate it at each point, we need exactly 1 squaring. For example, at $x = 2$ the polynomial equals $(4P + 2Q + R)^2$. So let's evaluate this polynomial at $-2, -1, 0, 1, 2$.

$$\begin{bmatrix} 16 & -8 & 4 & -2 & 1 \\ 1 & -1 & 1 & -1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 16 & 8 & 4 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} P^2 \\ 2PQ \\ Q^2 + 2PR \\ 2QR \\ R^2 \end{bmatrix} = \begin{bmatrix} (4P - 2Q + R)^2 \\ (P - Q + R)^2 \\ R^2 \\ (P + Q + R)^2 \\ (4P + 2Q + R)^2 \end{bmatrix}$$

The vector on the right hand side represents the 5 squarings we have to do. Now to solve x in $Ax = b$, since A is some fixed matrix just filled with constants, we could precompute A^{-1} , or we could do row operations if we want to just manipulate the linear equations we got. Either ways, we can solve for the coefficients i.e the vector x with few additions/subtractions. $\square$

Question 2. Can you find multiplication of two n -bit integers, using the multiplication subroutine on six pairs of $n/3$ bit integers and a few additions/subtractions? You need to compute the following five terms using the multiplication operation only six times.

$$a_0b_0, a_1b_0 + a_0b_1, a_0b_2 + a_1b_1 + a_2b_0, a_1b_2 + a_2b_1, a_2b_2.$$

Now, do this with only five multiplications.

Solution. If we are allowed 6 multiplications, first we just calculate a_0b_0 , a_1b_1 , and a_2b_2 . To compute $a_1b_0 + a_0b_1$, and $a_2b_1 + a_1b_2$, we do Karatsuba's trick and compute $(a_1 + a_0)(b_1 + b_0)$ and $(a_2 + a_1)(b_2 + b_1)$. For the middle term, since we already have a_1b_1 , we just need $a_2b_0 + a_0b_2$, so we just compute $(a_2 + a_0)(b_2 + b_0)$, and we're done. For 5 multiplications, we do what we did in the last question. We just need to find the coefficients of $(a_2x^2 + a_1x + a_0)(b_2x^2 + b_1x + b_0)$. For this we evaluate the polynomial at 5 different points and for each evaluation, we are multiplying 2 $n/3$ bit numbers. $\square$

Question 3. Can you find square of an n -bit integer a , using square subroutine on $2k - 1$ integers with n/k bits and some additions/subtractions? What's the running time you get? What if you take k as something like $n/2$? Does that give you a really fast algorithm?

Solution. We can do something similar to Q2. We just need to find the coefficients of $(\sum_{i=0}^{k-1} a_i x^i)^2$. This is a polynomial with degree $2k - 2$, so to find the coefficients, we need to evaluate it at $2k - 1$ points. For each evaluation, we'll have to square a n/k bit integers. So, this takes $(2k - 1)T(n/k)$ time.

But the other step that we have to do is now solve $2k - 1$ linear equations in $2k - 1$ variables. If we do the normal row operations, i.e Gaussian elimination, this takes $O((2k - 1)^3(n/k)) = O(k^2n)$. This is because each row might have to be subtracted from every row below it during elimination, and every row subtraction involves $2k - 1$ individual subtractions. The n/k factor comes because the addition/subtraction is happening with n/k bit integers. So our recursion is $T(n) = (2k - 1)T(n/k) + O(k^2n)$. Now we can do something similar to the previous question. Assume $n = k^c$.

$$\begin{aligned}
T(k^c) &= (2k - 1)T(k^{c-1}) + O(k^2(k^c)) \\
&= (2k - 1)((2k - 1)T(k^{c-2}) + O(k^2(k^{c-1}))) + O(k^{c+2}) \\
&= (2k - 1)^2 T(k^{c-2}) + O(k^{c+2} + (2k - 1)k^{c+1}) \\
&= (2k - 1)^3 T(k^{c-3}) + O(k^{c+2} + (2k - 1)k^{c+1} + (2k - 1)^2 k^c) \\
&\vdots \\
&= (2k - 1)^c T(1) + O(k^{c+2} + (2k - 1)k^{c+1} + \dots + (2k - 1)^{c-1} k^3) \\
&= (2k - 1)^c T(1) + O\left(k^3 \left(\frac{(2k - 1)^c - k^c}{(2k - 1) - k}\right)\right) \\
&= (2k - 1)^c T(1) + O(k^2((2k - 1)^c - k^c))
\end{aligned}$$

Since $2k - 1 > k$, the second term is $O(k^2(2k - 1)^c)$, and the first term is clearly $O((2k - 1)^c)$, so overall, $T(k^c) = O(k^2((2k - 1)^c))$. Substituting $c = \log_k n$, we get $T(n) = O(k^2((2k - 1)^{\log_k n})) = O(k^2 n^{\log_k(2k-1)})$.

If we take $k = n/2$, $T(n) = O((n/2)^2 n^{\log_{n/2}(n-1)})$, which is worse than $O(n^2)$. $\square$

Question 4. Matrix Multiplication. Let us say we have 8 numbers $a_1, a_2, a_3, a_4, b_1, b_2, b_3, b_4$ and we consider these seven expressions.

$$\begin{aligned} p_1 &= (a_1 + a_4)(b_1 + b_4), & p_2 &= (a_3 + a_4)b_1, \\ p_3 &= a_1(b_2 - b_4), & p_4 &= a_4(b_3 - b_1), \\ p_5 &= (a_1 + a_2)b_4, & p_6 &= (a_3 - a_1)(b_1 + b_2), \\ p_7 &= (a_2 - a_4)(b_3 + b_4) \end{aligned}$$

(a) Compute the following four sums. This will be helpful later.

$$p_1 + p_4 - p_5 + p_7, \quad p_3 + p_5, \quad p_2 + p_4, \quad p_1 - p_2 + p_3 + p_6$$

Now, we want to apply divide and conquer technique to matrix multiplication. Let A and B be two $n \times n$ matrices, and we want to compute their product $C = A \times B$. The naive algorithm for this will take $O(n^3)$ arithmetic operations. We want to significantly improve this using divide and conquer. A natural way to split any matrix can be this:

$$A = \begin{pmatrix} A_1 & A_2 \\ A_3 & A_4 \end{pmatrix}$$

where each A_i is an $n/2 \times n/2$ matrix.

(b) Can you express the product matrix C , in terms of A_1, A_2, A_3, A_4 and B_1, B_2, B_3, B_4 .

(c) Design an algorithm for matrix multiplication using divide and conquer which takes $O(7^{\log_2 n}) = O(n^{\log_2 7}) = O(n^{2.81})$ time.

Solution. (a) It's just some algebra, but at the end we get the 4 sums as $a_1b_1 + a_2b_3, a_1b_2 + a_2b_4, a_3b_1 + a_4b_3, a_3b_2 + a_4b_4$.

(b) Because of how matrix multiplication works, treating the submatrices as numbers and multiplying like 2×2 matrices actually gives us the multiplication of the matrices if done normally.

$$C = \begin{pmatrix} A_1B_1 + A_2B_3 & A_1B_2 + A_2B_4 \\ A_3B_1 + A_4B_3 & A_3B_2 + A_4B_4 \end{pmatrix}$$

(c) The divide and conquer is basically done from the previous parts. First we just divide A, B each into 4 submatrices, and we have 4 different terms to compute. But in order to compute these terms, we can compute the terms p_1 to p_7 but substituting submatrices instead of numbers. Note that each p_i will take exactly 1 matrix multiplication of size $n/2 \times n/2$. Now that the p_i 's are calculated, we can do few additions and subtraction as we did in the first part to get the 4 terms that we need, they are the same as the terms that we need in the second part. So this is how we get C . The recursion for this is that $T(n) = 7T(n/2) + O(n^2)$ ($O(n^2)$ for additions and subtractions). Solving this recursion gives $T(n) = O(n^{\log_2 7}) = O(n^{2.81})$. □

Question 5. Let $P(x) = a_0 + a_1x + \cdots + a_{d-1}x^{d-1}$ be a degree $d-1$ polynomial. Let the d -th roots of unity be $\omega^0, \omega^1, \dots, \omega^{d-1}$. Let the evaluations of $P(x)$ on the d -th roots of unity be $e_0, e_1, \dots, e_{d-1}$. That is, for each $0 \leq i \leq d-1$

$$e_i = P(\omega^i) = a_0 + a_1\omega^i + \cdots + a_{d-1}\omega^{(d-1)i}$$

Define a new polynomial $Q(y)$ as $e_0 + e_1x + \cdots + e_{d-1}x^{d-1}$. Prove that for each $0 \leq i \leq d-1$

$$Q(\omega^{-i}) = e_0 + e_1\omega^{-i} + \cdots + e_{d-1}\omega^{-(d-1)i} = da_i.$$

Hint: Useful facts to prove, $\omega^{-i} = \omega^{d-i}$. For any $1 \leq i \leq d-1$, $\sum_{j=0}^{d-1} \omega^{ij} = 0$.

Solution. $e_i = P(\omega^i) = a_0 + a_1\omega^i + a_2\omega^{2i} + \cdots + a_{d-1}\omega^{(d-1)i} = \sum_{j=0}^{d-1} a_j\omega^{ij}$. We can just expand each coefficients in the polynomial $Q(y)$ and evaluate using this.

$$\begin{aligned} Q(\omega^{-i}) &= e_0\omega^{-i} + e_1\omega^{-2i} + \cdots + e_{d-1}\omega^{-(d-1)i} \\ &= \sum_{k=0}^{d-1} e_k\omega^{-ik} \\ &= \sum_{k=0}^{d-1} \left(\sum_{j=0}^{d-1} a_j\omega^{kj} \right) \omega^{-ik} \\ &= \sum_{k=0}^{d-1} \sum_{j=0}^{d-1} a_j\omega^{(j-i)k} \end{aligned}$$

Let's now reorder the summation a bit, and group terms according to j .

$$\begin{aligned} Q(\omega^{-i}) &= \sum_{j=0}^{d-1} \sum_{k=0}^{d-1} a_j\omega^{(j-i)k} \\ &= \sum_{j=0}^{d-1} \left(\sum_{k=0}^{d-1} \omega^{(j-i)k} \right) a_j \end{aligned}$$

Now we just have some weighted sum of a_j 's, all we have to do is calculate their weights. Firstly let's try to find the weight of a_i . This weight is going to be $\sum_{k=0}^{d-1} \omega^{(i-i)k} = \sum_{k=0}^{d-1} 1 = d$, so 1 term is just da_i . Now take the case where $j \neq i$. The weight is now $\sum_{k=0}^{d-1} \omega^{(j-i)k} = \sum_{k=0}^{d-1} u^k$, where $u = \omega^{j-i}$. But we also know that u is a complex root of unity, so $1 + u + u^2 + \cdots + u^{d-1} = 0$, so the summation which is the weight, is basically 0. So the final answer we get is

$$Q(\omega^{-i}) = \sum_{j=0}^{d-1} \left(\sum_{k=0}^{d-1} \omega^{(j-i)k} \right) a_j = da_i$$

□

Question 6. Suppose we want to evaluate $P(x) = a_0 + a_1x + a_2x^2 + a_3x^3$ at the fourth roots of unity 1, -1, i , $-i$. Consider how FFT algorithm will compute these. We will represent the algorithm as a circuit, with the α gate shown in figure 1. The gate labeled with α takes two numbers a and b (in that order) as inputs and outputs two numbers $a + \alpha b$ and $a - \alpha b$ (in that order).

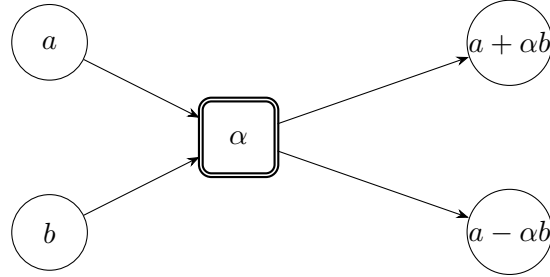


Figure 1: The operation that forms the building block of the FFT algorithm, represented as a gate labeled with a root of unity α

The FFT circuit shown in Figure 2 shows the computation for evaluations of a degree 3 polynomial at 4th roots of unity. Fill in the empty circles with the appropriate input values and intermediate values.

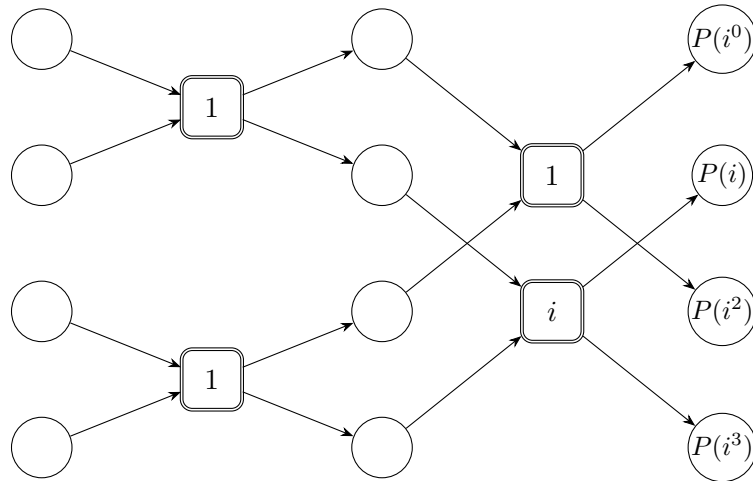


Figure 2: The FFT algorithm for a degree 3 polynomial $P(x)$ evaluated at 4th roots of unity

The FFT circuit shown in Figure 3 shows the computation for evaluations of a degree 7 polynomial at 8th roots of unity. Fill in the empty circles with the appropriate input values and intermediate values.

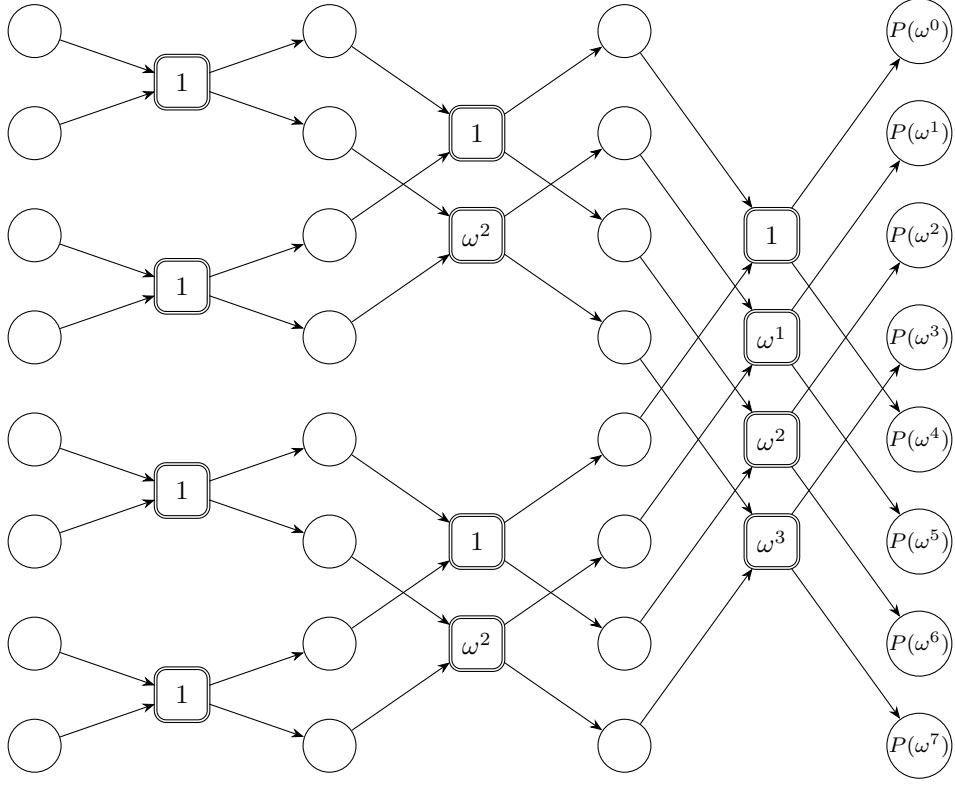
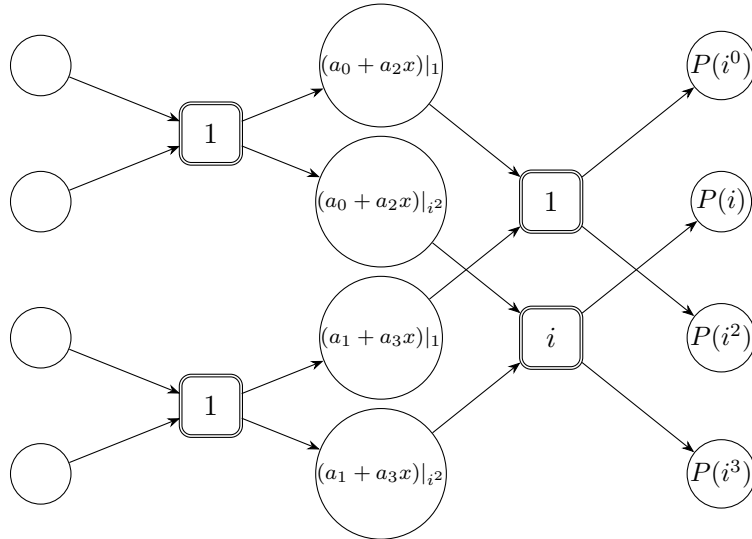
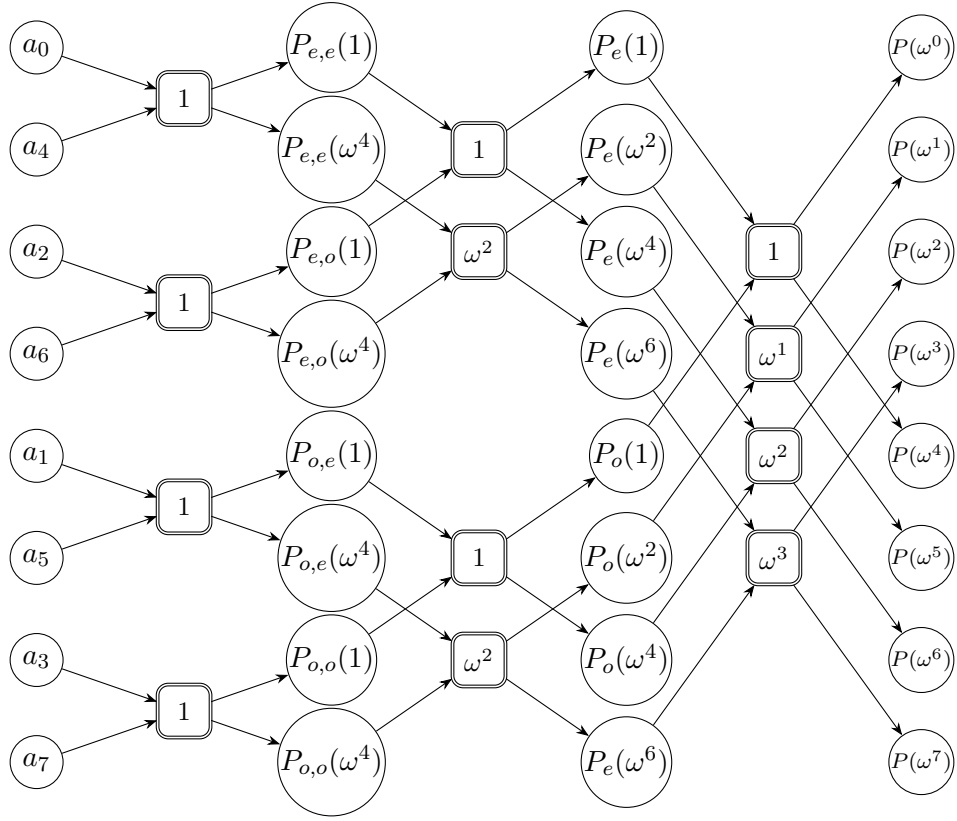


Figure 3: The FFT algorithm for a degree 7 polynomial $P(x)$ evaluated at 8th roots of unity. Here ω is the primitive 8th root of unity.

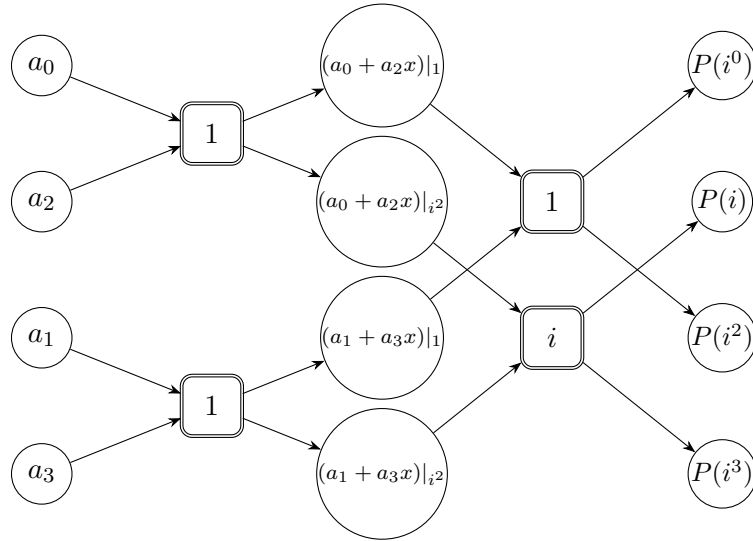
Solution. First, let us consider the evaluation of the degree 3 polynomial at the 4th roots of unity. To calculate $P(x) = a_0 + a_1x + a_2x^2 + a_3x^3$ at $1, -1, i$, and $-i$, we use the gate to combine expressions of the form $P_{\text{even}}(\alpha^2)$ and $P_{\text{odd}}(\alpha^2)$ to evaluate $P(\alpha)$ and $P(-\alpha)$ together (for $\alpha = 1, i$):

$$\begin{aligned} P(1) &= (a_0 + a_2x)|_1 + 1(a_1 + a_3x)|_1 & P(-1) &= (a_0 + a_2x)|_1 - 1(a_1 + a_3x)|_1 \\ P(i) &= (a_0 + a_2x)|_{-1} + i(a_1 + a_3x)|_{-1} & P(-i) &= (a_0 + a_2x)|_{-1} - i(a_1 + a_3x)|_{-1} \end{aligned}$$





The next step should therefore be to evaluate $(a_0 + a_2x)|_{1,-1}$ and $(a_1 + a_3x)|_{1,-1}$ using the gate with $\alpha = 1$.



We can use a similar backtracking argument to evaluate the degree 7 polynomial at the 8^{th} roots of unity. For brevity, $P_e(x) = (a_0 + a_2x + a_4x^2 + a_6x^3)$, $P_o(x) = (a_1 + a_3x + a_5x^2 + a_7x^3)$, $P_{e,e}(x) = (a_0 + a_4x)$, $P_{e,o}(x) = (a_2 + a_6x)$, $P_{o,e}(x) = (a_1 + a_5x)$, and $P_{o,o}(x) = (a_3 + a_7x)$.

□

Question 7. Interval scheduling: Suppose we have two rooms. And we have a set of requests each with a time interval. We want to allocate the rooms to maximum number of requests. That is, we want to find two subsets S_1, S_2 of intervals with $S_1 \cap S_2 = \emptyset$ such that each of two is a set of disjoint intervals and want to maximize $|S_1| + |S_2|$. Prove that the following greedy algorithm will give an optimal solution.

Initialize S_1 and S_2 as empty sets. t_1, t_2 will denote the largest finish times among the intervals in S_1 and S_2 , respectively. Initially $t_1 = t_2 = 0$. Go over the given intervals in increasing order of finish times. When we are at the i -th interval (a_i, b_i) ,

- if $a_i < t_1$ and $a_i < t_2$ then discard this interval
- if $a_i \geq t_1$ and $a_i < t_2$ then put this interval in S_1 .
- if $a_i < t_1$ and $a_i \geq t_2$ then put this interval in S_2 .
- if $a_i \geq t_1 \geq t_2$ then put this interval in S_1 .
- if $a_i \geq t_2 > t_1$ then put this interval in S_2 .

Prove the optimality of this greedy algorithm. One way to prove this is as follows. Assume that there is an optimal solution that agrees with the choices made by greedy algorithm on first $i - 1$ intervals. Modify this solution to get another optimal solution that agrees with the choices made by greedy algorithm on first i intervals. Then inductively, you can finish the proof.

Solution. We are given a set of intervals $I_1, \dots, I_n$, where $I_i = (a_i, b_i)$, sorted in increasing order of finish times. We have two rooms R_1 and R_2 , and we wish to schedule a maximum number of intervals so that no two overlapping intervals are assigned to the same room. Let G be the set of intervals selected by the greedy algorithm. We prove that G is optimal by showing that there exists an optimal solution O that matches every choice made by Greedy.

Note that the greedy algorithm processes intervals in increasing order of finish times. When an interval is feasible in both rooms, it assigns the interval to the room whose last assigned interval finishes later; if no room is feasible, the interval is discarded.

Inductive Hypothesis: Assume there exists an optimal solution O that agrees with the greedy algorithm on these intervals, both in terms of which intervals are selected and their room assignments for the first $i - 1$ intervals. Let t_1 and t_2 denote the finish times of the two rooms after these $i - 1$ steps. WLOG, assume $t_1 \geq t_2$.

Inductive Step: We use the argument provided in the question and show using case analysis that O can be modified to another optimal solution O' that agrees with greedy on intervals $I_1, I_2, \dots, I_i$.

1. **Case 1:** $a_i < t_1$ and $a_i < t_2$: Greedy discards I_i because it overlaps with the last intervals in both R_1 and R_2 . Since O agrees with Greedy on the first $i - 1$ intervals, picking I_i is not feasible for O . Therefore, O must also discard I_i .
2. **Case 2:** $a_i \geq t_1$ and $a_i \geq t_2$: Greedy assigns I_i to R_1 :
 - If O assigns I_i to R_2 : O reaches state (t_1, b_i) , while Greedy reaches (b_i, t_2) . Since $t_1 \geq t_2$, swapping O 's intervals after t_1 leaves us with the required O' that agrees with G on $I_1, \dots, I_i$.
 - If O discards I_i : Since I_i is feasible in R_1 , O must have used R_1 for a later interval I_k ($b_k \geq b_i$). We can replace I_k with I_i to get the required O' since b_i is the earliest finish time among remaining intervals and hence does not decrease the total count.

3. **Case 3:** $a_i \geq t_2$ and $a_i < t_1$ Greedy assigns I_i to R_2 . (O can't assign I_i to R_1 due to a reason similar to Case 1):

- If O discards I_i : Similar to the last problem, we can replace O 's next interval in R_2 with I_i to match Greedy.

By repeating this process for all n intervals, we transform any optimal solution into the greedy solution without reducing the total number of scheduled intervals. Thus, the greedy algorithm is optimal. $\square$

Question 8. Suppose you are given a set of n course assignments today, each of which has its own deadline. Let the i -th assignment have deadline d_i and suppose to finish the i -th assignment it takes l_i time. Given that there are so many assignments, it might not be possible to finish all of them on time. If you finish an assignment at time t_i which is more than its deadline d_i , then the difference $t_i - d_i$ is called the lateness of this assignment (if $t_i < d_i$ then the lateness is zero). Since you want to maintain a balance among courses, you want that the maximum lateness over all assignments is as small as possible. You want to find a schedule for doing the assignments which minimizes the maximum lateness over all assignments.

Example: $d_1 = 20, l_1 = 10, d_2 = 40, l_2 = 20, d_3 = 60, l_3 = 30$. If the assignments are done in order (1,3,2) then the maximum lateness will be 20 (for assignment 2). If the assignments are done in order (1,2,3) then the maximum lateness will be 0. Can you show that a greedy algorithm will give you an optimal solution?

- Greedy Strategy 1: Do the assignments in increasing order of their lengths (l_i).
- Greedy Strategy 2: Do that assignment first whose deadline is the closest.
- Greedy Strategy 3: Do that assignment first for which $d_i - l_i$ is the smallest.

Strategies 1 and 3 don't work. Give examples to show that they don't work.

Strategy 2 works. Prove that it works correctly as follows. Consider any particular schedule for the assignments. Pick any two assignments A_i and A_j which are adjacent, that is, A_j is done immediately after A_i . If $d_i \leq d_j$ do then nothing, but if $d_i > d_j$ then swap the positions of A_i and A_j . Argue that after this step, the maximum lateness cannot increase. Repeatedly use the same argument to show that increasing order of deadlines is an optimal schedule.

Solution. Strategy 1 fails in cases where you actually have less time for larger assignments. Take an example there $l_1, l_2 = 1, 2$ and $d_1, d_2 = 3, 2$. If you do 1 before 2, you finish 2 late, but if you do 1 after 2, you finish both on time.

Strategy 3 seems logical. The reason we are using $d_i - l_i$ as a factor, is because that's the time remaining after we finish the assignment. So it's an increasing order of how much extra time we will have for an assignment if we start now. But it can prioritize very long assignments over short ones which need to be finished early. Take $l_1, l_2 = 1, 3$ and $d_1, d_2 = 3, 4$. Our greedy approach will make us do assignment 2 first, which makes us finish assignment 1 late. But if we did assignment 1 first, we would have enough time for assignment 2 after it.

Now to prove the claim in strategy 2. Let's say A_j is immediately done after A_i , but A_j has a closer deadline compared to A_i . What happens if we swap A_i and A_j ? Firstly, it's clear that the lateness of any assignment other than A_i and A_j don't change. Now let's just look at A_i and A_j . We can take cases, but I'll provide some sort of general reasoning as to why the swap won't decrease the max lateness.

Before the swap, A_j was finished later, but had an earlier deadline. So no matter what,

the lateness will be more than or equal to A_i . So after swapping, we just have to make sure the lateness of both A_i, A_j are less than the lateness of A_j before swapping. Firstly after swapping the lateness of A_j obviously decreases, it's done earlier. But the lateness of A_i has increased. But think about it like this, A_i 's finishing time after swapping is same as A_j 's finishing time before swapping. And also, A_i 's deadline is further away than A_j 's deadline. So since the finishing times are equal and we have more time in 1 case, A_i 's lateness after swapping is $\leq A_j$'s lateness before swapping.

Now we can use this swapping trick to do the whole question. Assume there's an optimal solution which isn't increasing order of deadlines. We can just do an insertion sort like algorithm. And keep doing swaps to bring the array to increasing order of deadlines. We have also shown that in each swap, the max lateness isn't increasing, and it can't really decrease as we started with an optimal solution. So at the end of our insertion sort, the max lateness is same as that of the optimal solution, so increasing order of deadlines is always optimal. $\square$